INGENIERÍA EN SOFTWARE

# Code Smells clásicos

*Bloque 04 · Apunte completo de estudio*

Qué son · Las 5 familias · Código MAL y BIEN en C# · Refactoring seguro
Ejercicio integrador sobre el sistema de La Lomi de la Cañada

**Instituto Técnico Superior Córdoba**

Ingeniería en Software · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026


| # | Sección |
|---|---|
| 01 | ¿Qué es un code smell? · Conexión con lo que ya vieron |
| 02 | Bloaters: código que creció de más |
| 03 | Abusadores de la orientación a objetos |
| 04 | Smells que frenan el cambio |
| 05 | Prescindibles: lo que sobra |
| 06 | Acopladores: clases demasiado pegadas |
| 07 | Cómo detectarlos y refactorizar seguro |

# 01 · ¿Qué es un code smell?

Un **code smell** (olor de código) es una característica del código que **sugiere** un problema de diseño más profundo. El término lo popularizaron Kent Beck y Martin Fowler en el libro *Refactoring* (1999), que además propuso para cada olor el refactoring que lo cura.

La palabra clave es **sugiere**. Un smell no es un bug: el programa compila, los tests pasan y el cliente recibe su lomito. El problema aparece después, cuando hay que cambiar algo y cada cambio cuesta el triple de lo que debería.

> **Tip:** Pensalo como un olor raro en la cocina de la lomitería. No significa que la carne esté podrida, pero es una señal de que conviene revisar. A veces es solo el fernet que se volcó ayer. Otras veces es la heladera que no enfría. El olor no te da el diagnóstico: te dice **dónde mirar**.

## Smell, bug y deuda técnica

| Concepto | ¿Funciona? | ¿Qué es? | Ejemplo en la lomitería |
|---|---|---|---|
| **Bug** | No | Comportamiento incorrecto. | El sistema cobra dos veces el mismo lomito. |
| **Code smell** | Sí | Síntoma visible de un diseño que va a costar mantener. | Un método de 200 líneas que calcula, cobra, guarda e imprime. |
| **Deuda técnica** | Sí | El costo acumulado de todos esos atajos. | Agregar Mercado Pago lleva dos semanas en vez de dos días. |

En el apunte general vieron la deuda técnica como concepto. Los code smells son la forma **concreta** de encontrarla: si la deuda es lo que debemos, los smells son las facturas que aparecen arriba de la mesa.

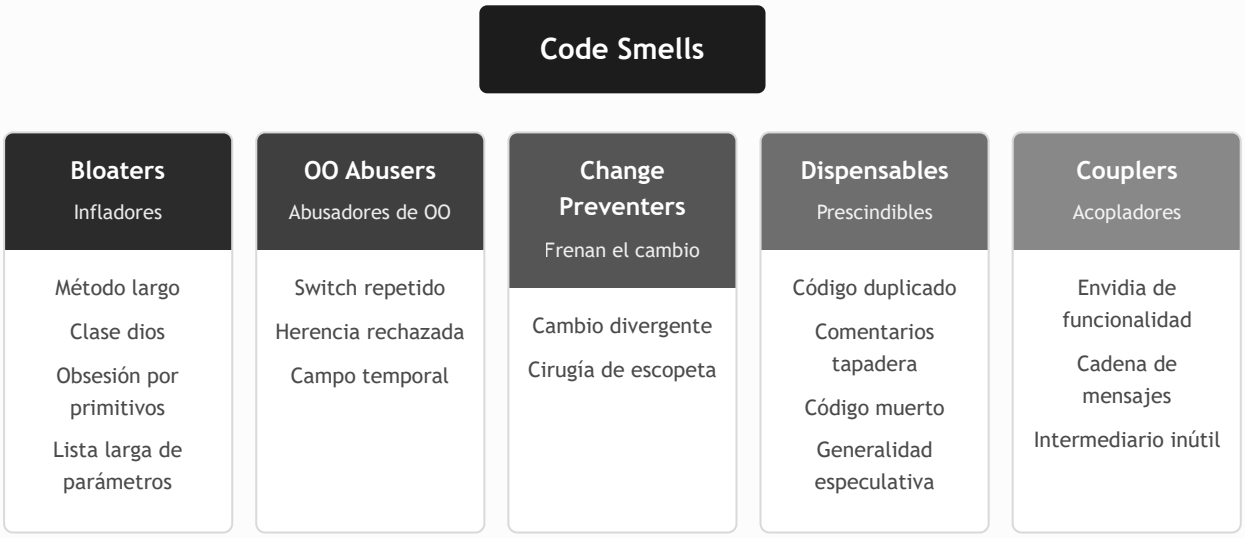
## Conexión con lo que ya vieron

| Ya lo vieron en… | Cómo se conecta con este bloque |
|---|---|
| **Deuda técnica y refactoring (apunte general)** | Los smells son los síntomas de la deuda. El refactoring es la forma de pagarla. |
| **Principios SOLID** | Casi todos los smells son una violación concreta de algún principio. Clase dios rompe SRP, switch repetido rompe OCP, herencia rechazada rompe LSP. |
| **Patrones de diseño** | Muchos refactorings terminan en un patrón: un switch por tipo de pago se cura con Strategy. |
| **Bloque 02 · Testing aplicado** | Los tests son la red de seguridad. Sin tests, refactorizar es adivinar. |

| Ya lo vieron en… | Cómo se conecta con este bloque |
|---|---|
| **Bloque 03 · Clean Architecture** | Es el destino. Refactorizar smells es el camino para pasar de un proyecto plano a uno ordenado. |
| **Programación III** | Controller, Service y Repository. Vamos a ver qué pasa cuando esas capas se mezclan. |

## Las cinco familias de smells

Existen decenas de smells catalogados. Para estudiarlos se suelen agrupar en cinco familias según **qué tipo de problema** generan. No hace falta memorizar los nombres en inglés, pero sí conviene reconocerlos porque así aparecen en la documentación y en las herramientas de análisis.




Las cinco familias y los smells que agrupa cada una.

| Familia | Problema que generan | Pregunta para detectarlos |
|---|---|---|
| **Bloaters** | Código que creció tanto que ya no se puede entender de una. | ¿Esto entra en una pantalla? ¿Lo puedo explicar en una frase? |
| **Abusadores de OO** | Se usa la orientación a objetos a medias o mal. | ¿Estoy usando if y switch donde podría usar polimorfismo? |
| **Frenan el cambio** | Un cambio chico obliga a tocar muchos lugares. | Si cambio esta regla de negocio, ¿cuántos archivos toco? |
| **Prescindibles** | Cosas que sobran y solo agregan ruido. | Si borro esto, ¿alguien lo extraña? |
| **Acopladores** | Clases que saben demasiado unas de otras. | ¿Esta clase usa más datos ajenos que propios? |

# 02 · Bloaters: código que creció de más

Los bloaters no aparecen de un día para el otro. Nadie escribe un método de 200 líneas a propósito: empieza con 20, alguien agrega el descuento para estudiantes, otro agrega el recargo del delivery, otro el ticket para la cocina… y un año después nadie se anima a tocarlo.

## Método largo (Long Method)

| | |
|---|---|
| **Categoría** | Bloater |
| **Cómo se reconoce** | El método no entra en la pantalla. Tiene comentarios que separan bloques ( `// validar` , `// calcular` ). Para entenderlo hay que hacer scroll. |
| **Refactoring que lo cura** | **Extract Method:** cada bloque comentado se convierte en un método con nombre propio. |
| **Qué principio rompe** | SRP: el método hace muchas cosas distintas. |

MAL

```
public decimal ProcesarPedido(Pedido pedido)
{
    // validar
    if (pedido == null) throw new ArgumentException("Pedido nulo");
    if (pedido.Lineas.Count == 0) throw new ArgumentException("Pedido sin ítems");

    // calcular subtotal
    decimal subtotal = 0;
    foreach (var linea in pedido.Lineas)
        subtotal += linea.Precio * linea.Cantidad;

    // aplicar descuento
    if (pedido.Cliente.EsEstudiante) subtotal *= 0.85m;

    // calcular envío
    decimal envio = pedido.EsDelivery ? 1500 : 0;
    if (pedido.DistanciaKm > 5) envio += 800;

    // guardar
    _context.Pedidos.Add(pedido);
    _context.SaveChanges();

    // avisar a cocina
    _impresora.Imprimir($"Pedido #{pedido.Id}: {pedido.Lineas.Count} ítems");

    return subtotal + envio;
}
```

**BIEN**

```
public decimal ProcesarPedido(Pedido pedido)
{
    Validar(pedido);
    var total = AplicarDescuento(pedido.Cliente, CalcularSubtotal(pedido))
              + CalcularEnvio(pedido);
    Guardar(pedido);
    AvisarACocina(pedido);
    return total;
}
// Cada método privado tiene 3 a 6 líneas y un nombre que explica qué hace.
```

> **Tip:** Los comentarios que separan bloques son **nombres de métodos esperando nacer**. Si escribiste `// calcular envío`, ya tenés el nombre: `CalcularEnvio()`.

# Clase dios (Large Class / God Class)

| | |
|---|---|
| **Categoría** | Bloater |
| **Cómo se reconoce** | Una clase con decenas de métodos y campos que no tienen relación entre sí. Su nombre suele terminar en *Manager*, *Helper*, *Utils* o *General*. |
| **Refactoring que lo cura** | **Extract Class:** agrupar métodos y campos por responsabilidad y moverlos a clases nuevas. |
| **Qué principio rompe** | SRP y alta cohesión. |

**MAL**

```
public class LomiteriaManager
{
    public void TomarPedido(Pedido p) { ... }
    public void CancelarPedido(int id) { ... }
    public decimal Cobrar(Pedido p, string metodo) { ... }
    public void CerrarCaja() { ... }
    public void DescontarStock(string ingrediente, int cant) { ... }
    public bool HayStock(string ingrediente) { ... }
    public void AltaEmpleado(Empleado e) { ... }
    public void LiquidarSueldos() { ... }
    public byte[] GenerarReporteVentasPdf(DateTime desde) { ... }
    public void EnviarPromoPorWhatsapp(string mensaje) { ... }
    // ... y 40 métodos más
}
```

**BIEN**

```
public class PedidoService   { /* TomarPedido, CancelarPedido */ }
public class CajaService     { /* Cobrar, CerrarCaja */ }
public class StockService    { /* DescontarStock, HayStock */ }
public class EmpleadoService { /* AltaEmpleado, LiquidarSueldos */ }
public class ReporteService  { /* GenerarReporteVentasPdf */ }
public class MarketingService{ /* EnviarPromoPorWhatsapp */ }
```

La prueba rápida: si para describir la clase necesitás la palabra **"y"** ("maneja los pedidos y la caja y el stock y los sueldos"), tiene más de una razón para cambiar.

## Obsesión por primitivos (Primitive Obsession)

| | |
|---|---|
| **Categoría** | Bloater |
| **Cómo se reconoce** | Conceptos del negocio representados con `string`, `int` o `decimal` sueltos. Estados como texto, teléfonos como string, montos sin moneda. |
| **Refactoring que lo cura** | **Replace Primitive with Object** (crear un tipo propio o value object) y **Replace Type Code with Enum**. |
| **Qué principio rompe** | Encapsulamiento: la validación queda desparramada en lugar de vivir en el tipo. |

**MAL**

```
public class Pedido
{
    public string Estado { get; set; }          // "recibido", "listo", "lsito"...
    public string TelefonoCliente { get; set; } // "351-555 1234"? "3515551234"? "hola"?
    public decimal Total { get; set; }           // ¿con IVA? ¿sin IVA?
}

if (pedido.Estado == "Listo") { ... } // nunca entra: se guardó "listo" en minúscula
```

**BIEN**

```
public enum EstadoPedido { Recibido, EnPreparacion, Listo, Entregado, Cancelado }

public record Telefono
{
    public string Numero { get; }
    public Telefono(string numero)
    {
        if (!Regex.IsMatch(numero, @"^\d{10}$"))
            throw new ArgumentException("El teléfono debe tener 10 dígitos");
        Numero = numero;
    }
}

public class Pedido
{
    public EstadoPedido Estado { get; private set; }
    public Telefono TelefonoCliente { get; private set; }
}

if (pedido.Estado == EstadoPedido.Listo) { ... } // el compilador te cuida
```

> **Conexión:** ¿Se acuerdan del diagrama de estados del pedido del Bloque 01? Ese diagrama **es** el enum. Si el estado es un string, el diagrama no lo hace cumplir nadie.

# Lista larga de parámetros y grupos de datos (Data Clumps)

| | |
|---|---|
| **Categoría** | Bloater |
| **Cómo se reconoce** | Métodos con más de 3 o 4 parámetros. Grupos de datos que siempre viajan juntos (calle, número, barrio, ciudad) y aparecen repetidos en varias firmas. |
| **Refactoring que lo cura** | **Introduce Parameter Object** y **Preserve Whole Object:** pasar el objeto completo en lugar de sus partes. |
| **Qué principio rompe** | Cohesión: datos que pertenecen juntos están separados. |

**MAL**

```
public void RegistrarDelivery(string nombre, string telefono, string calle, int numero,
    string barrio, string ciudad, decimal total, bool pagaEnEfectivo,
    decimal pagaCon, string observaciones) { ... }

// Llamada: ¿cuál es el número y cuál el total? ¿true es efectivo?
RegistrarDelivery("Juan", "3515551234", "Obispo Trejo", 242, "Nueva Córdoba",
    "Córdoba", 18500, true, 20000, "sin cebolla");
```

**BIEN**

```
public record Direccion(string Calle, int Numero, string Barrio, string Ciudad);
public record DatosPago(MetodoPago Metodo, decimal? PagaCon = null);

public void RegistrarDelivery(Cliente cliente, Direccion destino,
    DatosPago pago, string? observaciones = null) { ... }
```

Un efecto secundario muy bueno: una vez que existe `Direccion`, aparece el lugar natural para poner lógica que antes estaba perdida, como `direccion.EstaEnZonaDeReparto()`.

# 03 · Abusadores de la orientación a objetos

Estos smells aparecen cuando el lenguaje es orientado a objetos pero el código se piensa de forma procedural: muchos `if`, muchos `switch` y herencia usada para reutilizar código aunque no tenga sentido.

## Switch repetido (Switch Statements)

| | |
|---|---|
| **Categoría** | Abusador de OO |
| **Cómo se reconoce** | El mismo `switch` o cadena de `if / else if` sobre un tipo (método de pago, tipo de cliente, tipo de producto) aparece en varios lugares. |
| **Refactoring que lo cura** | **Replace Conditional with Polymorphism.** Suele terminar en el patrón **Strategy**. |
| **Qué principio rompe** | OCP: cada método de pago nuevo obliga a modificar todos los switch. |

**MAL**

```
public decimal CalcularRecargo(string metodoPago, decimal total)
{
    switch (metodoPago)
    {
        case "efectivo":    return 0;
        case "debito":      return 0;
        case "credito":     return total * 0.10m;
        case "mercadopago": return total * 0.06m;
        default: throw new ArgumentException("Método de pago desconocido");
    }
}
// El mismo switch aparece en Facturar(), en ReporteCaja() y en ImprimirTicket().
// Agregar "transferencia" = buscar y modificar los cuatro.
```

**BIEN**

```
public interface IMetodoPago
{
    string Nombre { get; }
    decimal CalcularRecargo(decimal total);
}

public class PagoEfectivo : IMetodoPago
{
    public string Nombre => "Efectivo";
    public decimal CalcularRecargo(decimal total) => 0;
}

public class PagoCredito : IMetodoPago
{
    public string Nombre => "Crédito";
    public decimal CalcularRecargo(decimal total) => total * 0.10m;
}
// Agregar transferencia = crear PagoTransferencia. No se toca nada existente.
```

> **Ojo:** Un switch **aislado** no es un smell. Muchas veces hace falta uno solo, por ejemplo en una factory que decide qué `IMetodoPago` crear. El olor aparece cuando el **mismo** switch se repite en varios lugares.

# Herencia rechazada (Refused Bequest)

| | |
|---|---|
| **Categoría** | Abusador de OO |
| **Cómo se reconoce** | Una subclase hereda métodos que no puede o no quiere usar. Los sobrescribe vacíos o con `throw new NotImplementedException()`. |
| **Refactoring que lo cura** | **Replace Inheritance with Delegation** o separar la jerarquía con interfaces más chicas. |
| **Qué principio rompe** | LSP (la subclase no puede reemplazar a la base) e ISP. |

**MAL**

```
public abstract class Producto
{
    public string Nombre { get; init; }
    public abstract int MinutosDeCoccion();
}

public class Lomito : Producto
{
    public override int MinutosDeCoccion() => 12;
}

public class Fernetcola : Producto
{
    public override int MinutosDeCoccion()
        => throw new NotImplementedException(); // ¿cocinar un fernet?
}
```

**BIEN**

```
public abstract class Producto { public string Nombre { get; init; } }
public interface ICocinable { int MinutosDeCoccion(); }

public class Lomito : Producto, ICocinable
{
    public int MinutosDeCoccion() => 12;
}

public class Fernetcola : Producto { } // no promete lo que no puede cumplir
```

Es exactamente el ejemplo del pingüino que hereda de `Ave` en el apunte de SOLID, pero con productos de la lomitería. Si la cocina recorre todos los productos y llama a `MinutosDeCoccion()`, la fernetcola rompe el sistema.

# Campo temporal (Temporary Field)

| | |
|---|---|
| **Categoría** | Abusador de OO |
| **Cómo se reconoce** | Campos de la clase que solo tienen valor durante un método puntual. El resto del tiempo están en `null` o en cero. |
| **Refactoring que lo cura** | **Extract Class** o convertir el campo en variable local o parámetro. |
| **Qué principio rompe** | Cohesión: el estado de la clase no representa al objeto. |

**MAL**

```
public class CajaService
{
    private decimal _totalTemporal;  // solo se usa dentro de CerrarCaja()
    private int _cantidadTickets;    // idem

    public void CerrarCaja() { _totalTemporal = 0; _cantidadTickets = 0; ... }
}
```

Si alguien llama a otro método de `CajaService` mientras se cierra la caja, esos campos tienen valores a medio calcular. Como variables locales de `CerrarCaja()` o dentro de un objeto `CierreDeCaja`, el problema desaparece.

# 04 · Smells que frenan el cambio

Estos son los más caros porque no se ven mirando una sola clase: aparecen cuando hay que hacer un cambio. Son dos caras de la misma moneda.

| | Cambio divergente (Divergent Change) | Cirugía de escopeta (Shotgun Surgery) |
|---|---|---|
| **Síntoma** | Una clase cambia por **muchos** motivos distintos. | **Un** cambio obliga a tocar **muchas** clases. |
| **Ejemplo** | Se modifica `PedidoService` cuando cambia la BD, cuando cambian los precios y cuando cambia el formato del ticket. | Sube el IVA y hay que buscar `1.21m` en siete archivos distintos. |
| **Causa** | Demasiadas responsabilidades juntas. | Una responsabilidad desparramada. |
| **Cura** | Extract Class: separar por motivo de cambio. | Move Method / Move Field: juntar lo desparramado en un solo lugar. |
| **Principio** | SRP. | DRY y alta cohesión. |

## Cirugía de escopeta en acción

**MAL**

```
// En PedidoService
var total = subtotal * 1.21m;
// En FacturaService
var importeFinal = neto * 1.21m;
// En ReporteVentas
var conIva = ventas.Sum(v => v.Monto) * 1.21m;
// En TicketCocina, en DeliveryService, en Presupuesto...
```

**BIEN**

```
public interface IImpuestoService
{
    decimal AplicarIva(decimal neto);
}

public class ImpuestoService : IImpuestoService
{
    private const decimal ALICUOTA_IVA = 0.21m;
    public decimal AplicarIva(decimal neto) => neto * (1 + ALICUOTA_IVA);
}
// Si mañana cambia la alícuota, o hay alícuotas distintas por producto,
// se toca un solo archivo.
```

> **Tip:** La cirugía de escopeta suele venir acompañada de **números mágicos**: valores como `1.21m`, `1500` o `0.85m` escritos directo en el código. Si el número tiene significado de negocio, merece un nombre.

La forma de detectarlos en la práctica es mirar el historial de Git: si cada commit sobre una funcionalidad toca diez archivos que no tienen nada que ver entre sí, hay cirugía de escopeta. Si un mismo archivo aparece en commits de temas totalmente distintos, hay cambio divergente.

# 05 · Prescindibles: lo que sobra

Son elementos que, si se borraran, el código quedaría más claro sin perder nada. Son los más fáciles de arreglar y los que más rápido se acumulan.

## Código duplicado (Duplicated Code)

| | |
|---|---|
| **Categoría** | Prescindible |
| **Cómo se reconoce** | El mismo bloque (o casi el mismo) aparece en dos o más lugares. Suele nacer del copy-paste. |
| **Refactoring que lo cura** | **Extract Method** si está en la misma clase; **Pull Up Method** si está en clases hermanas; **Extract Class** si está en clases sin relación. |
| **Qué principio rompe** | DRY. |

MAL

```
// En PedidoController
decimal total = 0;
foreach (var l in pedido.Lineas) total += l.Precio * l.Cantidad;
if (pedido.Cliente.EsEstudiante) total *= 0.85m;

// En DeliveryService (copiado, pero alguien olvidó el descuento)
decimal total = 0;
foreach (var l in pedido.Lineas) total += l.Precio * l.Cantidad;
```

El problema real del duplicado no es que ocupa más líneas: es que **las copias divergen**. Cuando se corrige un error en una, la otra queda con el error. En el ejemplo, los pedidos por delivery nunca reciben el descuento de estudiante y nadie sabe por qué.

## Comentarios tapadera (Comments)

| | |
|---|---|
| **Categoría** | Prescindible |
| **Cómo se reconoce** | Comentarios que explican **qué** hace un código porque el código es ilegible. |
| **Refactoring que lo cura** | **Extract Method** y **Rename:** que el nombre diga lo que decía el comentario. |
| **Qué principio rompe** | Legibilidad y mantenibilidad (ISO 25010). |

**MAL**

```
// Si el cliente es estudiante, pidió más de 2 lomitos y es martes, aplica promo
if (c.T == 2 && p.L.Count(x => x.Tp == 1) > 2 &&
    DateTime.Now.DayOfWeek == DayOfWeek.Tuesday)
    p.Tot *= 0.8m;
```

**BIEN**

```
if (promoMartes.Aplica(cliente, pedido))
    pedido.AplicarDescuento(promoMartes.Porcentaje);
```

> **Ojo:** No todos los comentarios son malos. Los que explican **por qué** se tomó una decisión son valiosos: `// ARCA rechaza montos con más de 2 decimales, por eso redondeamos acá`. El smell es el comentario que explica **qué** hace un código que debería explicarse solo.

## Código muerto y generalidad especulativa

| | Código muerto (Dead Code) | Generalidad especulativa (Speculative Generality) |
|---|---|---|
| **Qué es** | Métodos que nadie llama, variables que no se usan, bloques comentados "por las dudas". | Abstracciones, parámetros o clases creadas "por si algún día" se necesitan. |
| **Ejemplo** | `// var ticket = GenerarTicketViejo(p);` comentado hace un año. | `IExportador` con implementaciones PDF, XML, CSV y JSON cuando solo se usa PDF. |
| **Cura** | Borrarlo. Git lo recuerda si alguna vez hace falta. | Collapse Hierarchy, Inline Class: sacar la abstracción que sobra. |
| **Principio** | KISS. | YAGNI. |

El argumento "lo dejo comentado por si lo necesito" no tiene sentido cuando el proyecto está en Git. El historial guarda todo. El código comentado solo confunde al que lo lee después.

# 06 · Acopladores: clases demasiado pegadas

El objetivo de diseño siempre es **alta cohesión y bajo acoplamiento**. Estos smells son los síntomas de lo contrario: clases que saben demasiado de otras, o que existen solo para pasar mensajes.

## Envidia de funcionalidad (Feature Envy)

| | |
|---|---|
| **Categoría** | Acoplador |
| **Cómo se reconoce** | Un método usa más datos de **otra** clase que de la propia. Accede a cinco propiedades de un objeto ajeno para calcular algo. |
| **Refactoring que lo cura** | **Move Method:** llevar el método a la clase dueña de los datos. |
| **Qué principio rompe** | Encapsulamiento y cohesión. Es el principio *Tell, don't ask*. |

**MAL**

```
public class CajaService
{
    public decimal CalcularTotal(Pedido p)
    {
        decimal total = 0;
        foreach (var l in p.Lineas) total += l.Precio * l.Cantidad;
        if (p.TieneCupon) total -= p.Cupon.Monto;
        if (total < 0) total = 0;
        return total;
    }
}
```

**BIEN**

```
public class Pedido
{
    public decimal CalcularTotal()
    {
        var total = Lineas.Sum(l => l.Subtotal());
        if (TieneCupon) total -= Cupon.Monto;
        return Math.Max(total, 0);
    }
}
// CajaService ahora solo le pide al pedido su total:
var total = pedido.CalcularTotal();
```

> **Conexión:** En Clean Architecture (Bloque 03) vimos que la lógica de negocio pura vive en **Domain**. Mover `CalcularTotal()` a la entidad `Pedido` es exactamente eso: el cálculo depende solo de datos del pedido, entonces le pertenece al pedido.

# Cadena de mensajes (Message Chains)

| | |
|---|---|
| **Categoría** | Acoplador |
| **Cómo se reconoce** | Expresiones largas del tipo `a.B.C.D.E`. El que llama conoce toda la estructura interna de los objetos intermedios. |
| **Refactoring que lo cura** | **Hide Delegate:** que el primer objeto exponga directamente lo que se necesita. |
| **Qué principio rompe** | Ley de Demeter: hablá solo con tus amigos cercanos. |

**MAL**

```
var costoEnvio = pedido.Cliente.Direccion.Barrio.Zona.TarifaDelivery;
// Si mañana Zona se calcula por código postal, esta línea (y todas las parecidas) se rompen.
```

**BIEN**

```
var costoEnvio = pedido.CostoDeEnvio();
```

No confundir con las cadenas de LINQ: `.Where(...).OrderBy(...).Take(10)` no es un smell, porque cada método devuelve el mismo tipo de colección y no se está navegando la estructura interna de objetos distintos.

# Intimidad inapropiada (Inappropriate Intimacy)

| | |
|---|---|
| **Categoría** | Acoplador |
| **Cómo se reconoce** | Una clase accede a campos internos de otra o modifica su estado directamente, salteando sus métodos. |
| **Refactoring que lo cura** | **Move Method**, **Encapsulate Field** o **Encapsulate Collection**. |
| **Qué principio rompe** | Encapsulamiento. |

**MAL**

```
// La cocina descuenta stock modificando la lista interna del StockService
_stockService.Ingredientes.First(i => i.Nombre == "pan").Cantidad -= 1;
```

**BIEN**

```
_stockService.Descontar("pan", 1); // StockService decide cómo, valida y registra
```

# Intermediario inútil (Middle Man)

| | |
|---|---|
| **Categoría** | Acoplador |
| **Cómo se reconoce** | Una clase cuyos métodos solo delegan en otra clase, sin agregar nada. |
| **Refactoring que lo cura** | **Remove Middle Man:** llamar directamente a la clase que hace el trabajo. |
| **Qué principio rompe** | KISS. |

**¿SMELL O NO?**

```
public class ProductoService : IProductoService
{
    private readonly IProductoRepository _repo;
    public ProductoService(IProductoRepository repo) => _repo = repo;

    public Task<List<Producto>> GetAll()      => _repo.GetAll();
    public Task<Producto?> GetById(int id)    => _repo.GetById(id);
    public Task Add(Producto p)               => _repo.Add(p);
    public Task Delete(int id)                => _repo.Delete(id);
}
```

> **Ojo:** Este caso genera debate y es una buena pregunta de parcial. En Programación III armaron Controller, Service y Repository, y muchas veces el Service solo delega. ¿Es un smell? **Depende de si la capa tiene un propósito.** Si la arquitectura define que la lógica de negocio vive en el Service (y mañana va a validar stock antes de borrar un producto), la capa se justifica aunque hoy sea fina. Si se agregó solo porque sí y nunca va a tener lógica, es un intermediario inútil. Un smell es una señal para pensar, no una regla automática.

# 07 · Cómo detectarlos y refactorizar seguro

## Tabla resumen

| Smell | Se reconoce porque… | Refactoring | Principio |
|---|---|---|---|
| **Método largo** | No entra en la pantalla, tiene bloques comentados. | Extract Method | SRP |
| **Clase dios** | Su descripción necesita la palabra "y". | Extract Class | SRP |
| **Obsesión por primitivos** | Estados como string, montos y teléfonos sueltos. | Replace Primitive with Object · Enum | Encapsulamiento |
| **Lista larga de parámetros** | Más de 3 o 4 parámetros. Datos que viajan juntos. | Introduce Parameter Object | Cohesión |
| **Switch repetido** | El mismo switch en varios lugares. | Replace Conditional with Polymorphism | OCP |
| **Herencia rechazada** | Métodos heredados que lanzan NotImplementedException. | Delegation · interfaces chicas | LSP · ISP |
| **Campo temporal** | Campos que casi siempre están en null. | Extract Class | Cohesión |
| **Cambio divergente** | Una clase cambia por muchos motivos. | Extract Class | SRP |
| **Cirugía de escopeta** | Un cambio toca muchos archivos. | Move Method · Move Field | DRY |
| **Código duplicado** | Bloques iguales o casi iguales. | Extract Method · Pull Up | DRY |
| **Comentarios tapadera** | El comentario explica qué hace el código. | Extract Method · Rename | Legibilidad |
| **Código muerto** | Código comentado o que nadie llama. | Borrar | KISS |
| **Generalidad especulativa** | Abstracciones "por si algún día". | Inline Class · Collapse Hierarchy | YAGNI |
| **Envidia de funcionalidad** | Usa más datos ajenos que propios. | Move Method | Tell, don't ask |
| **Cadena de mensajes** | `a.B.C.D.E` | Hide Delegate | Demeter |

| Smell | Se reconoce porque… | Refactoring | Principio |
|---|---|---|---|
| **Intimidad inapropiada** | Modifica el estado interno de otra clase. | Encapsulate Field | Encapsulamiento |
| **Intermediario inútil** | Solo delega, sin propósito. | Remove Middle Man | KISS |

## Valores orientativos

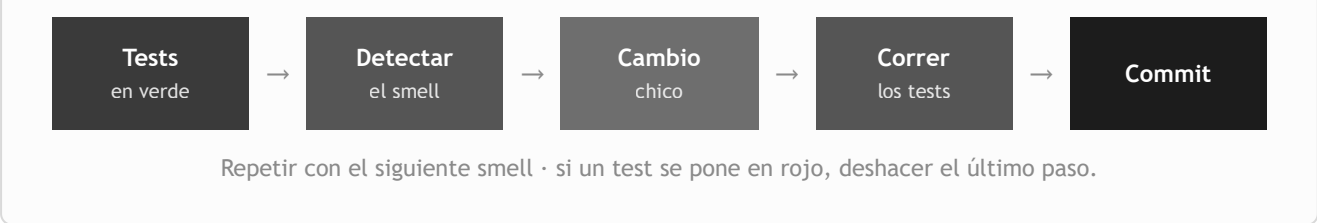
No existen números exactos, pero estas referencias sirven para levantar la mano y preguntar "¿esto está bien?". Superarlas no es automáticamente un error.

| Medida | Empezá a sospechar si… | Herramienta que lo mide |
|---|---|---|
| **Largo de un método** | Supera 20 a 30 líneas. | Métricas de código de Visual Studio |
| **Parámetros** | Son más de 3 o 4. | Analizadores de Roslyn, SonarQube |
| **Complejidad ciclomática** | Supera 10 en un método (muchos if, switch y bucles anidados). | Métricas de código de Visual Studio |
| **Largo de una clase** | Supera las 300 líneas o tiene más de 20 métodos públicos. | SonarQube, Rider |
| **Duplicación** | Bloques de más de 5 líneas repetidos. | SonarQube |

En Visual Studio se accede desde **Analizar > Calcular métricas de código**. Muestra el índice de mantenibilidad, la complejidad ciclomática, el acoplamiento de clases y las líneas de código por proyecto, clase y método. SonarQube (y su extensión para el IDE) detecta muchos smells automáticamente y se puede sumar al pipeline de GitHub Actions que armaron en el primer período.

## Refactoring seguro

Refactorizar es cambiar la estructura interna **sin cambiar el comportamiento**. La única forma de saber que el comportamiento no cambió es tener tests. Por eso el Bloque 02 vino antes que este.




- **Primero los tests.** Si el código no tiene tests, el primer paso es escribirlos para el comportamiento actual, aunque ese comportamiento sea feo.

- **Pasos chicos.** Un Extract Method, correr tests. Un Rename, correr tests. Nunca diez cambios juntos.
- **Un commit por paso.** Si algo se rompe, `git revert` te devuelve al último estado bueno.
- **Los dos sombreros.** Kent Beck propone que nunca se refactoriza y se agrega funcionalidad al mismo tiempo. O estás con el sombrero de refactorizar, o con el de agregar features. Nunca los dos.
- **Usá el IDE.** Visual Studio y Rider tienen refactorings automáticos (Ctrl + . en Visual Studio) para extraer métodos, renombrar o mover clases. Son más seguros que hacerlo a mano.

> **Tip:** La **regla del boy scout**: dejá el código un poco más limpio de lo que lo encontraste. No hace falta refactorizar todo el sistema: si tocás un método para agregar algo, aprovechá y renombrá esa variable `t` que nadie entiende.

# 08 · Ejercicio integrador

El siguiente endpoint de La Lomi de la Cañada **funciona**: crea pedidos y los guarda. Encontrá al menos **ocho** code smells, nombralos, indicá en qué línea o parte del código aparecen y qué refactoring aplicarías.

**EJERCICIO · PedidosController.cs**

```
[ApiController]
[Route("api/[controller]")]
public class PedidosController : ControllerBase
{
    private readonly AppDbContext _db;
    public PedidosController(AppDbContext db) => _db = db;

    [HttpPost]
    public IActionResult Crear(string nom, string tel, string calle, int num,
        string barrio, string metodo, List<int> prods, bool deliv)
    {
        // validar
        if (nom == null || nom == "") return BadRequest();
        if (tel.Length != 10) return BadRequest();

        decimal t = 0;
        foreach (var id in prods)
        {
            var p = _db.Productos.Find(id);
            t += p.Precio;
        }

        // recargo
        if (metodo == "credito") t = t * 1.10m;
        else if (metodo == "mercadopago") t = t * 1.06m;

        if (deliv)
        {
            t += 1500;
            if (barrio != "Nueva Córdoba" && barrio != "Güemes") t += 800;
        }

        t = t * 1.21m; // IVA

        var ped = new Pedido { Cliente = nom, Telefono = tel, Total = t, Estado = "recibido" };
        _db.Pedidos.Add(ped);
        _db.SaveChanges();

        // var ticket = GenerarTicketViejo(ped);
        // _impresora.Imprimir(ticket);

        return Ok(ped);
    }
}
```

# Solución orientativa

*Intentá resolverlo antes de leer esta tabla.*

| Smell | Dónde aparece | Refactoring |
|---|---|---|
| **Método largo** | Todo `Crear()`: valida, calcula, aplica recargos, guarda. | Extract Method: Validar, CalcularSubtotal, CalcularRecargo, CalcularEnvio. |
| **Lista larga de parámetros** | Ocho parámetros en la firma. | Introduce Parameter Object: un `CrearPedidoDto` recibido con `[FromBody]`. |
| **Data clump** | `calle, num, barrio` siempre juntos. | Record `Direccion`. |
| **Obsesión por primitivos** | `tel` string, `Estado = "recibido"`, `metodo` string. | Value object `Telefono`, enum `EstadoPedido`. |
| **Switch repetido (if encadenados)** | Recargo según `metodo`. | Strategy con `IMetodoPago`. |
| **Números mágicos** | `1.10m, 1.06m, 1500, 800, 1.21m`. | Constantes con nombre o `IImpuestoService`. |
| **Nombres sin significado** | `t, nom, tel, num, deliv, p, ped`. | Rename. |
| **Comentarios tapadera** | `// validar, // recargo, // IVA`. | Extract Method con esos nombres. |
| **Código muerto** | Las dos líneas comentadas del ticket. | Borrarlas: Git las recuerda. |
| **Clase con responsabilidades mezcladas** | El controller tiene lógica de negocio y accede a `_db` directo. | Mover lógica a `PedidoService` y acceso a datos a un repositorio (Prog III, Bloque 03). |

> **Conexión:** Además de los smells, hay dos problemas que vieron en Programación III: `Find()` puede devolver `null` y el código revienta con un 500, y el método es sincrónico cuando debería usar `async / await` con `FindAsync()` y `SaveChangesAsync()`. No son code smells, son bugs y malas prácticas, pero un buen code review los marcaría igual.

# Para pensar en clase

- ¿En qué orden aplicarías los refactorings? ¿Por cuál empezarías y por qué?
- ¿Qué tests escribirías **antes** de tocar una sola línea?

- Si tuvieras que agregar "transferencia" como método de pago, ¿cuántas líneas tocarías en el código original y cuántas en el código refactorizado?
- ¿Cómo quedaría la estructura de carpetas de esta funcionalidad siguiendo Clean Architecture?

# Glosario del bloque

| Término | Definición |
|---|---|
| **Code smell** | Característica del código que sugiere un problema de diseño. No es un error, es una señal. |
| **Refactoring** | Cambiar la estructura interna del código sin cambiar su comportamiento externo. |
| **Bloater** | Familia de smells de código que creció demasiado: métodos, clases o firmas enormes. |
| **Número mágico** | Valor literal con significado de negocio escrito directo en el código, sin nombre. |
| **Value object** | Objeto definido por su valor y no por su identidad. Se valida al crearse. Ej: `Telefono`, `Direccion`. |
| **Complejidad ciclomática** | Cantidad de caminos independientes que tiene un método. Cada if, case o bucle suma uno. |
| **Ley de Demeter** | Un objeto solo debería hablar con sus colaboradores directos, no con los colaboradores de sus colaboradores. |
| **Tell, don't ask** | Pedirle al objeto que haga algo con sus datos, en vez de pedirle los datos y hacerlo afuera. |
| **Regla del boy scout** | Dejar el código un poco más limpio de lo que se encontró. |
| **Los dos sombreros** | Nunca refactorizar y agregar funcionalidad en el mismo paso. |